SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	R KALAI AMUDHAN	**Reg. No.:**	192511292
Date:		**Duration:**	60 minutes

Code of Conduct

I R KALAI AMUDHAN (192511292) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R KALAI AMUDHAN

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.

5. **Version Control Workflow**
    - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
    - Explain the purpose of each Git operation performed.
6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

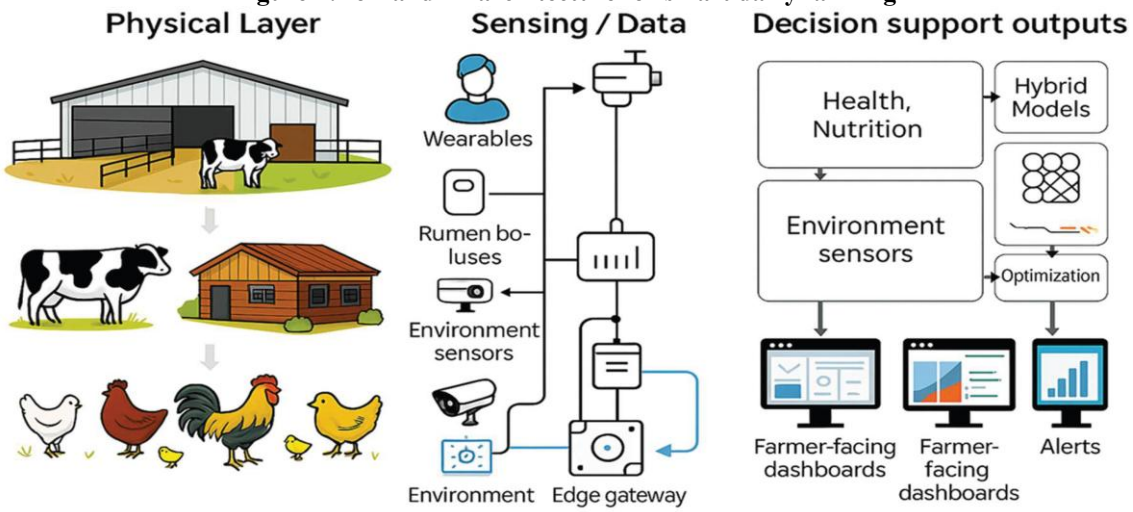
GIT & DOCKER TECHNICAL ASSIGNMENT

Intelligent AI-Based Smart Dairy Farm Management System

1. Problem Analysis

1.1 Problem Statement

Figure 1: IoT and AI architecture for smart dairy farming



Modern dairy farms require continuous monitoring of livestock health, milk production, feeding schedules, and environmental conditions. Traditional dairy farm management relies heavily on manual observation, making it difficult to detect animal health problems early, optimize feeding schedules, and maintain production efficiency.

The proposed Intelligent AI-Based Smart Dairy Farm Management System uses IoT sensors, Artificial Intelligence, cloud computing and a web-based dashboard to monitor cattle health, predict diseases, optimize feeding schedules, track milk production and generate farm performance reports.

1.2 Project Objectives

1. Monitor cattle health continuously using wearable and environmental sensors.
2. Predict possible livestock diseases using an AI prediction module.
3. Optimize feeding schedules according to cattle data and farm requirements.
4. Track daily milk production and identify production trends.
5. Monitor environmental conditions such as temperature and humidity.
6. Generate farm performance reports for management decisions.
7. Improve animal welfare and reduce avoidable health problems.
8. Reduce operational costs and improve dairy productivity.

1.3 Functional Requirements

- User registration, login and role-based access.
- Cattle registration and profile management.
- Continuous health-data monitoring.
- AI-based disease prediction.
- Environmental-condition monitoring.
- Alerts and notifications for abnormal conditions.
- Farm performance reports.

1.4 Expected Outcomes

- Improved livestock health.
- Increased milk production.
- Reduced disease outbreaks.
- Better farm management.
- Lower operational costs.
- Increased farmer profitability.

2. Project Structure Design

The repository is organized into separate application, database, testing and deployment files so development, version control and containerization can be managed independently.

```
smart-dairy-farm/
├── app/
│   ├── __init__.py
│   ├── routes.py
│   ├── models.py
│   ├── ai_model.py
│   ├── templates/
│   └── static/
├── tests/
│   └── test_app.py
├── database/
│   └── schema.sql
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── README.md
└── run.py
```

2.1 Purpose of Major Files and Folders

File / Folder	Purpose
app/	Main Flask application.
templates/	HTML user-interface pages.
static/	CSS, JavaScript and images.
ai_model.py	Disease-prediction logic.
models.py	Application/database models.
database/	SQL schema and database scripts.
tests/	Application test cases.
requirements.txt	Python dependencies.
Dockerfile	Defines how the application container is built.
docker-compose.yml	Defines and connects application services.
.gitignore	Prevents unwanted files from being committed.
.dockerignore	Excludes unnecessary files from Docker build context.
README.md	Project setup and usage documentation.
run.py	Starts the Flask application.

3. Git Repository Initialization

Git is used to track source-code changes, maintain project history and support collaborative development.

3.1 Repository Initialization Steps

1. Create the project folder.
2. Open a terminal inside the project folder.
3. Initialize a local Git repository.
4. Configure the Git user name and email.
5. Create the initial project files.
6. Stage the files.
7. Create the first commit.
8. Connect the local repository to GitHub and push the project.

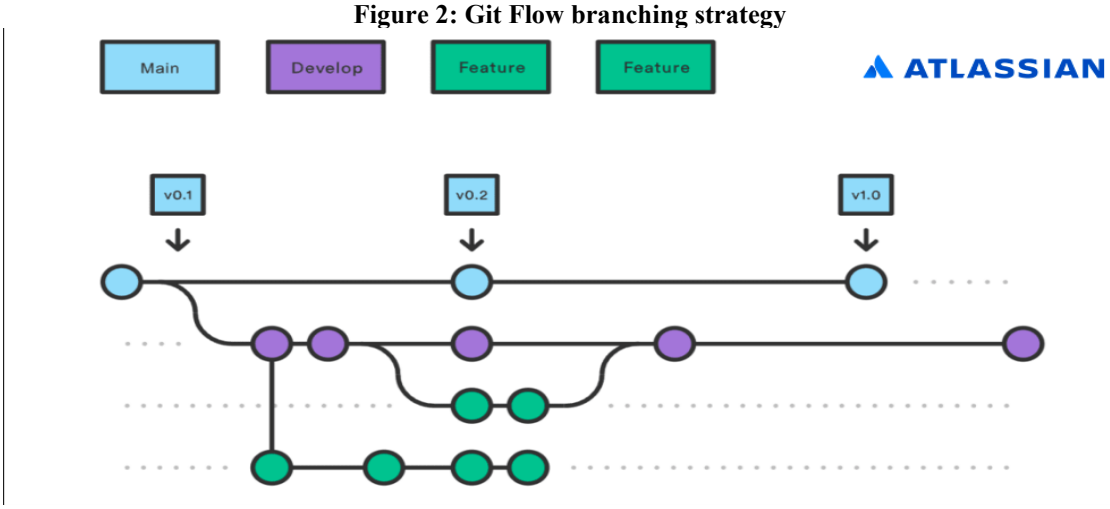
```
mkdir smart-dairy-farm
cd smart-dairy-farm
git init
git config user.name "Your Name"
git config user.email "your@email.com"
git add .
git commit -m "Initial project setup"
git branch -M main
git remote add origin <GitHub-Repository-URL>
git push -u origin main
```

3.2 Essential Git Files

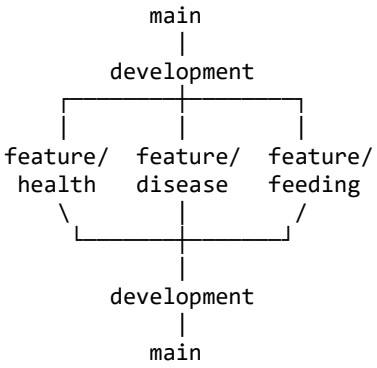
• .git/ stores Git repository metadata and commit history.
• .gitignore lists files that should not be tracked.
• README.md documents the project and setup instructions.
• .dockerignore controls which files are excluded from the Docker build context.

4. Git Branching Strategy

Figure 2: Git Flow branching strategy



A feature-based branching strategy with main and development branches is suitable because the system contains multiple independent modules such as health monitoring, disease prediction, feeding and milk tracking.



Branch	Purpose
main	Stable production-ready version.
development	Integration and testing branch.
feature/health-monitoring	Cattle health monitoring module.
feature/disease-prediction	AI disease prediction module.
feature/feeding	Feeding management module.
feature/milk-tracking	Milk production tracking module.

4.1 Justification

The branching model isolates new features from the stable version, reduces accidental changes to main, supports parallel development and makes testing easier before release.

5. Version Control Workflow

Figure 3: Git feature-branch workflow

5.1 Git Operations

```
git status
git checkout -b feature/health-monitoring
git add .
git commit -m "Add cattle health monitoring"
git checkout development
git merge feature/health-monitoring
git pull
git push
```

Operation	Purpose
git status	Shows changed, staged and untracked files.
git checkout -b	Creates and switches to a new branch.
git add	Stages changes for the next commit.
git commit	Stores a meaningful version of the changes.
git merge	Combines a completed branch with another branch.
git pull	Downloads and integrates remote changes.
git push	Uploads local commits to the remote repository.

5.2 Conflict Resolution

If two branches modify the same lines, Git may report a merge conflict. The developer should inspect the marked conflict, keep the correct combined version, save the file, stage it and commit the resolved result.

```
git status
# edit the conflicted file
git add .
git commit -m "Resolve merge conflict in health module"
```

6. Commit History Analysis

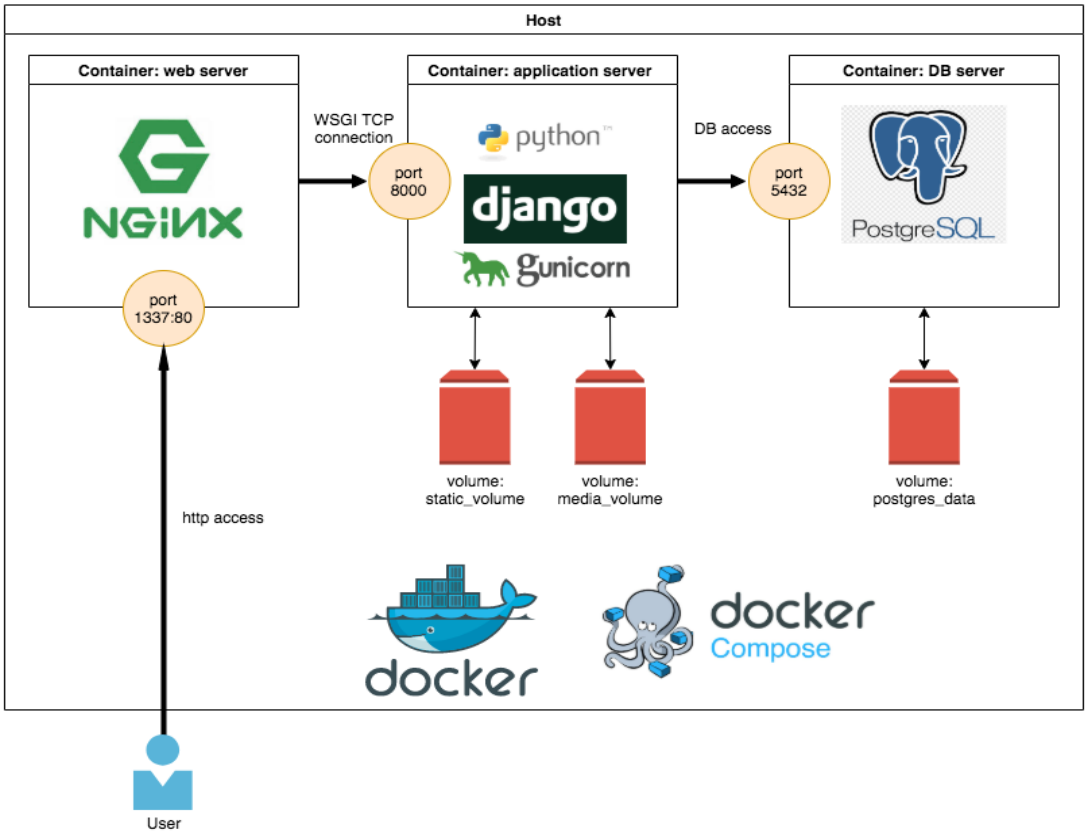
Commit messages should describe one logical change clearly and use action-oriented wording. This makes the history useful for maintenance, debugging and rollback.

```
git log --oneline --graph --all
```

• Initial project setup
• Add cattle registration module
• Add health monitoring module
• Implement AI disease prediction
• Add feeding management
• Add milk production tracking
• Add dashboard analytics
• Add Docker configuration
• Fix database connection
• Update project documentation

7. Docker Environment Design

Figure 4: Multi-container application architecture using Docker Compose



Docker is suitable because it packages the application and its dependencies into reproducible containers. The same environment can be used for development, testing and deployment, reducing configuration differences between machines.

Component	Container / Service	Purpose
Flask Web Application	web	Runs the backend, routes and dashboard.
MySQL Database	db	Stores cattle, health, feeding and milk records.
AI Module	Inside web service	Processes health data and performs disease prediction.

```
IoT / Sensor Data
        |
        v
Flask Web Application ----> AI Prediction Module
        |
        v
      MySQL
        |
        v
Web / Mobile Dashboard
        |
        v
Farmer / Farm Manager
```

8. Dockerfile Development

The Dockerfile defines the base image, working directory, dependencies, application files, exposed port and startup command.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "run.py"]
```

Instruction	Purpose
FROM	Selects a lightweight Python base image.
WORKDIR	Sets the working directory inside the container.
COPY	Copies project files into the image.
RUN	Installs required Python packages.
EXPOSE	Documents the Flask application port.
CMD	Starts the Flask application.

9. Docker Compose Design

```
services:
  web:
    build: .
    container_name: smart_dairy_app
    ports:
      - "5000:5000"
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_USER: dairyuser
      DB_PASSWORD: dairypass
      DB_NAME: dairyfarm

  db:
    image: mysql:8.0
    container_name: smart_dairy_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: dairyfarm
      MYSQL_USER: dairyuser
      MYSQL_PASSWORD: dairypass
    ports:
      - "3306:3306"
```

9.1 Services, Networking and Dependencies

The web service builds the Flask application from the Dockerfile. The db service uses MySQL 8.0. Docker Compose provides a network so the web service can reach MySQL using the service name db instead of localhost.

9.2 Environment Variables

- DB_HOST identifies the database service.
- DB_USER identifies the application database user.
- DB_PASSWORD provides the database password.
- DB_NAME identifies the application database.
- MYSQL_DATABASE creates the initial database.

10. Container Deployment and Testing

10.1 Build and Start

```
docker compose build
docker compose up -d
docker ps
```

10.2 Logs and Shutdown

```
docker compose logs
docker compose logs web
docker compose down
```

10.3 Testing Procedure

1. Build the application image using Docker Compose.
2. Start the web and database containers.
3. Confirm both containers are running with docker ps.
4. Open the Flask application at http://localhost:5000.
5. Test login and dashboard access.
6. Check cattle records and health information.
7. Verify milk production and feeding information.
8. Verify communication with MySQL.
9. Inspect Docker logs for errors.
10. Stop the services after testing.

10.4 Evidence to Attach

• Dockerfile screenshot.
• docker-compose.yml screenshot.
• docker compose build screenshot.
• docker compose up -d screenshot.
• docker ps screenshot showing running containers.
• Browser screenshot of the application dashboard.
• docker compose logs screenshot.
• GitHub repository and commit-history screenshot.

11. Benefits of Git and Docker

Area	Git Benefit	Docker Benefit
Collaboration	Branches and merges support parallel work.	Same application environment can be shared.
Version Management	Changes can be tracked and restored.	Reproducible deployment environments.
Portability	Source can be cloned to another machine.	Application and dependencies run consistently.
Deployment	Stable branches can hold release-ready code.	Containers simplify deployment.
Scalability	Feature branches support organized development.	Services can be scaled independently.
Maintainability	History helps debugging and rollback.	Dependencies and runtime are isolated.

12. Challenges and Improvements

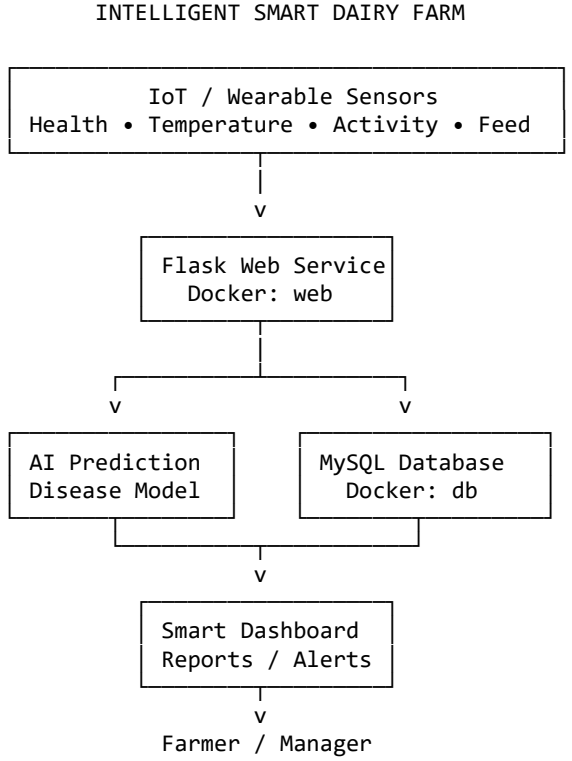
12.1 Challenges

• Maintaining correct dependencies in requirements.txt.
• Connecting Flask to MySQL inside the Docker network.
• Handling database startup readiness.
• Managing AI model size and container build time.
• Handling real-time sensor data reliably.
• Resolving Git merge conflicts.
• Keeping configuration and credentials outside source code.

12.2 Future Improvements

• Add CI/CD using GitHub Actions.
• Add Docker health checks and automatic restart policies.
• Deploy to a cloud platform.
• Use Kubernetes for large-scale orchestration.
• Add automated database backup and recovery.
• Improve AI disease prediction with more training data.
• Add real-time IoT data streaming.
• Add mobile notifications for urgent cattle-health alerts.
• Use secure secrets management for production credentials.

Architecture and Workflow Summary



13. System Architecture and Component Interaction

The Intelligent AI-Based Smart Dairy Farm Management System consists of several interconnected components that work together to provide intelligent farm management. The main components include IoT sensors, the Flask web application, AI prediction module, MySQL database, dashboard, and farmer interface.

13.1 IoT Sensor Layer

IoT and wearable sensors collect information from cattle and the farm environment. The collected data can include body temperature, activity level, movement, feeding behaviour, and environmental conditions.

13.2 Application Layer

The Flask application acts as the main communication layer between the user interface, AI module, and database. It receives requests from users and processes farm information.

13.3 AI Prediction Layer

The AI module analyzes collected cattle-health information and identifies abnormal patterns. Based on the available training data, it can support early disease-risk prediction.

13.4 Database Layer

MySQL stores cattle information, health records, feeding schedules, milk production data, sensor readings, alerts, and reports.

13.5 User Interface Layer

The dashboard provides farmers and farm managers with an easy way to view cattle information, health status, milk production, feeding schedules, alerts, and reports.

14. Data Flow and Communication

The system follows a continuous data flow between sensors, application services, database, AI module, and users.

14.1 Data Collection

Sensors collect cattle health and environmental information.

14.2 Data Transmission

The collected information is transmitted to the application system through the available communication network.

14.3 Data Processing

The Flask application receives and processes the sensor information. Relevant data is passed to the AI module for analysis.

14.4 AI Analysis

The AI module analyzes the available health parameters and identifies possible abnormal conditions or disease risks.

14.5 Database Storage
Processed information is stored in the MySQL database for future analysis and report generation.

14.6 Dashboard Display
The processed information is displayed on the dashboard so that the farmer can make appropriate decisions.

14.7 Alert Generation
If abnormal health conditions are identified, the system can generate an alert for the farmer or farm manager.

15. Security and Data Protection
Security is an important requirement for the Smart Dairy Farm Management System because the application stores farm information and user credentials.

15.1 User Authentication
Users should log in using valid credentials before accessing the system.

15.2 Role-Based Access
Different users can be given different permissions. For example, a farm manager may have access to reports and management functions, while other users may have limited access.

15.3 Database Security
Database credentials should not be directly exposed in source-code files. Environment variables or secure configuration methods should be used.

15.4 Git Security
Sensitive information such as passwords, API keys, database credentials, and private configuration files should be added to .gitignore and should not be uploaded to GitHub.

15.5 Docker Security
Only the required ports and services should be exposed. Production deployments should use secure secrets management instead of storing passwords directly in configuration files.

16. Testing Strategy
Testing ensures that the different components of the system work correctly.

16.1 Functional Testing
Functional testing verifies whether the application features work according to the requirements.
Examples:

- User login
- Cattle registration
- Health monitoring
- Disease prediction
- Feeding management
- Milk production tracking
- Report generation

16.2 Integration Testing
Integration testing verifies communication between different components.
Examples:

- Flask application ↔ MySQL
- Flask application ↔ AI module
- Dashboard ↔ Flask backend
- Docker web container ↔ MySQL container

16.3 Docker Testing
Docker deployment should be tested using:
docker compose build
docker compose up -d
docker ps
docker compose logs

16.4 Git Testing
Git operations should also be verified by checking:
git status
git branch
git log --oneline
git remote -v

17. Performance and Scalability
The system should be capable of supporting the increasing amount of data generated by a growing dairy farm.

17.1 Performance
The application should process sensor information efficiently and provide dashboard information without unnecessary delays.

17.2 Database Scalability
The database should be designed to handle increasing cattle records, health records, milk-production records, and sensor data.

17.3 Application Scalability
Docker makes it easier to deploy additional application containers when the number of users or requests increases.

17.4 AI Scalability
The AI module can be improved by training models with larger and more diverse datasets.

17.5 Future Cloud Deployment
The containerized application can later be deployed to a cloud environment to provide access from different locations.

18. GitHub Repository and Documentation

The GitHub repository acts as the central location for maintaining the project source code.
The repository should contain:
README.md
requirements.txt
Dockerfile
docker-compose.yml
.gitignore
.dockerignore
app/
database/
tests/

18.1 README Documentation

The README file should explain:

- Project title
- Problem statement
- Objectives
- Technology stack
- Project structure
- Installation steps
- Git commands
- Docker commands
- Application usage
- Testing procedure
- Future improvements

18.2 Repository Maintenance

The repository should be regularly updated with meaningful commits. Feature branches should be merged only after testing.

19. Continuous Integration and Continuous Deployment (CI/CD)

CI/CD can be introduced in the future to automate software development and deployment.

19.1 Continuous Integration

Whenever developers push changes to the repository, automated processes can:

1. Download the source code.
2. Install dependencies.
3. Run tests.
4. Check code quality.
5. Build the Docker image.
6.

19.2 Continuous Deployment

After successful testing, the application can automatically be deployed to a suitable server or cloud platform.

19.3 Benefits

- Faster development
- Automated testing
- Fewer deployment errors
- Consistent builds
- Faster bug detection
- Easier software updates

20. Project Maintenance and Monitoring

Continuous maintenance is required after deployment.

20.1 Application Monitoring

The application should be monitored for errors, performance problems, and service availability.

20.2 Docker Monitoring

Running containers can be monitored using:
docker ps
docker stats
docker compose logs

20.3 Database Maintenance

Regular backups should be performed to prevent loss of important farm information.

20.4 AI Model Maintenance

The AI model should periodically be evaluated and updated when new and reliable training data becomes available.

20.5 Git Maintenance

Unused branches should be removed after successful merging, while important releases should be properly tagged.

21. Overall Benefits of the Proposed System

The proposed system provides benefits to farmers, farm managers, animals, and the overall dairy business.

For Farmers

- Easier farm monitoring
- Real-time information
- Reduced manual work
- Better decision-making

For Cattle

- Continuous health monitoring
- Earlier identification of health problems
- Improved feeding management
- Better welfare

For Farm Management

- Milk production analysis
- Performance reports
- Reduced operational costs
- Better resource management

For the Business

- Increased productivity
- Reduced losses
- Better planning
- Improved profitability

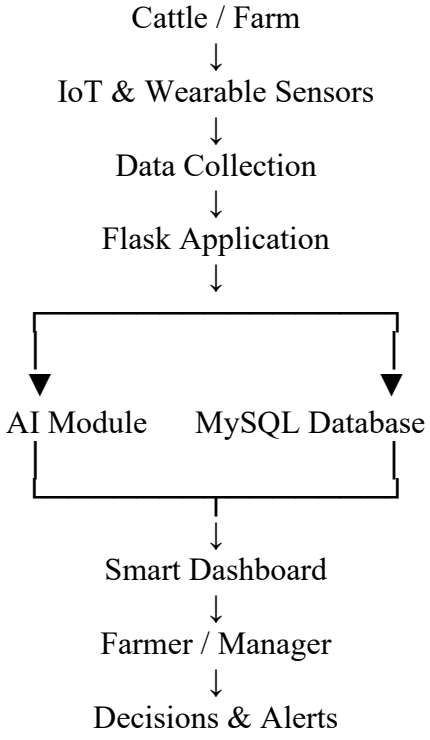
22. Future Scope

The system can be extended with several advanced technologies in the future.

1. Integration with advanced wearable cattle sensors.
2. Real-time mobile notifications.
3. Advanced AI disease prediction.
4. Automated robotic feeding systems.
5. Automatic milk-quality analysis.
6. Cloud-based farm management.
7. Computer vision for cattle monitoring.
8. Predictive milk-production analytics.
9. Automated vaccination reminders.
10. Multi-farm management from a single dashboard.
11. Advanced IoT data streaming.
12. Kubernetes-based container orchestration for large-scale deployment.

23. Final System Workflow

The complete system can be represented as:



Conclusion

The Intelligent AI-Based Smart Dairy Farm Management System provides a modern and efficient approach to dairy farm management by integrating AI, IoT sensors, cloud-based technologies, Git, and Docker. The system can continuously monitor cattle health, support disease prediction, manage feeding schedules, track milk production, and provide useful farm analytics.

The use of Git and GitHub helps maintain proper version control, track project changes, manage different development branches, and support collaborative software development. Docker and Docker Compose provide a consistent and portable environment in which the Flask application, AI components, and MySQL database can work together reliably.

Overall, the proposed system can reduce dependence on manual monitoring, improve decision-making, enhance animal welfare, and support better dairy productivity. In the future, the system can be further improved by integrating real-time IoT data, advanced AI disease prediction, cloud deployment, automated CI/CD, mobile notifications, and scalable container orchestration. These improvements can make the platform more reliable, scalable, and suitable for real-world smart dairy farm operations.